

Respuestas TP Interdisciplinario

Integrantes:Hermann Mayea, Angel Sierra, Tiago Bustos, Gian Arguello.

Docente: Fabrizio Nahuel Romero.

Año: 4 División:1

1. Estructuras de Datos en Pandas

a. DataFrame vs. Array Bidimensional

Un **DataFrame de Pandas** es una estructura de datos tabular de dos dimensiones (filas y columnas) que combina las ventajas de un array bidimensional de NumPy con funcionalidades adicionales para el análisis de datos.

Se puede pensar como una “hoja de cálculo” en memoria, donde cada columna puede tener un **tipo de dato diferente**, algo que los arrays de NumPy no permiten (en NumPy, todos los elementos del array deben ser del mismo tipo).

(La indexación es el mecanismo que permite acceder, identificar o seleccionar elementos dentro de una estructura de datos (como una lista, un array o un DataFrame).

En palabras simples, el índice es como una “etiqueta” o “dirección” que te dice dónde está cada dato y te permite recuperarlo, modificarlo o filtrarlo)

Característica	DataFrame (Pandas)	Array Bidimensional (NumPy o lista de listas)
Indexación	Basada en etiquetas (nombre de filas y columnas).	Basada en posición (índices numéricos).
Tipos de datos	Soporta tipos heterogéneos (numéricos, texto, fechas, booleanos, etc.) por columna.	Homogéneo: todos los elementos deben ser del mismo tipo.
Operaciones vectorizadas	Implementadas internamente con NumPy (muy eficientes).	Limitadas; suelen requerir bucles explícitos.
Manejo de datos faltantes	Usa NaN y funciones como fillna() , dropna() .	No maneja fácilmente valores faltantes.
Etiquetas y metadatos	Permite nombrar filas y columnas, mejorando la legibilidad.	No incluye etiquetas.

Aplicación práctica:

En un trabajo práctico de análisis de datos (por ejemplo, calificaciones de estudiantes), usar `DataFrame` permite acceder fácilmente a columnas como `df['puntaje']`, filtrar datos (`df[df['puntaje'] > 90]`), o calcular promedios (`df['puntaje'].mean()`), todo sin recorrer manualmente la estructura.

b. Series vs. Array Unidimensional

Una **Serie (Series)** en Pandas es una estructura **unidimensional**, similar a un array de NumPy o a una columna de un DataFrame.

Internamente, una **Series se construye sobre un objeto ndarray de NumPy**, pero añade un **índice (index)** que permite identificar cada elemento por etiqueta.

Estructura	Es...	Tiene índice	Es parte de...	Para qué sirve
Series	Columna con nombre	Si	Panda	Datos tabulados (como excel)
ndarray	Arreglo rápido	No	Numpy	Cálculo matemático rápido

Ejemplo práctico:

```
import pandas as pd
import numpy as np

serie = pd.Series([80, 90, 100], index=['Ana', 'Luis', 'Marta'])
array = np.array([80, 90, 100])

print(serie['Luis']) # Acceso por etiqueta
print(array[1])      # Acceso por índice
```

2. Algoritmos de Procesamiento (Vectorización)

Vectorización (NumPy/Pandas) vs. Iteración (Bucles for)

Primero voy a aclarar lo que es iterar y la vectorización:

Iterar

Iterar es básicamente repetir una acción sobre cada elemento de algo.

En programación significa:

“Recorrer esta lista/tabla/ o lo que sea, paso por paso”.

Vectorización

En pocas palabras:

Vectorizar es hacer operaciones sobre *muchos datos a la vez*, sin usar un bucle.

Es decir:

En vez *“Recorrer cada numero uno por uno y le sumo 10”*,

Es como *“Atento, yo lo hago todo junto en una sola operación y rápido, sin bucles explícitos”*.

En lugar de procesar elemento por elemento con **for**, las operaciones se ejecutan directamente sobre arrays completos, aprovechando que **NumPy y Pandas están implementados en C**, un lenguaje mucho más rápido.

Por ejemplo, para calcular el promedio de la columna **puntaje**:

```
# Vectorizado
promedio = df['puntaje'].mean()
```

vs.

```
# Iterativo
suma = 0
for p in df['puntaje']:
    suma += p
promedio = suma / len(df['puntaje'])
```

El método vectorizado es **mucho más rápido**, porque:

- La operación se ejecuta directamente en código compilado (C), sin pasar por el intérprete de Python.

- Se minimiza la sobrecarga de bucles e interpretación de instrucciones.
- NumPy realiza operaciones en bloque usando optimizaciones de bajo nivel y SIMD (procesamiento paralelo).

Aplicación práctica:

En análisis de grandes volúmenes de datos (por ejemplo, miles de calificaciones o registros de ventas), la vectorización permite obtener resultados en milisegundos, mientras que los bucles podrían tardar segundos o minutos.

3. Algoritmos de Ordenamiento y Búsqueda en el Análisis

a. Eficiencia en la Búsqueda

Cuando ejecutamos una instrucción como:

```
df[df['puntaje'] > 90]
```

Se realiza un **filtrado booleano** o **búsqueda condicional**.

El proceso:

1. Se evalúa la expresión `df['puntaje'] > 90`, que devuelve una **Serie booleana** (True/False por fila).
2. Pandas aplica esa máscara para seleccionar solo las filas donde la condición es verdadera.

Este tipo de búsqueda es **vectorizada**: Pandas evalúa la condición para toda la columna usando NumPy en C, sin iterar fila por fila, lo que la hace mucho más eficiente que un bucle `for`.

Complejidad aproximada: $O(n)$, ya que recorre la columna una sola vez para aplicar la condición.

b. Ordenamiento

Cuando usamos:

```
df.sort_values('puntaje', ascending=False)
```

Pandas ordena el DataFrame según los valores de la columna `puntaje`.

Internamente, Pandas delega el ordenamiento a NumPy, que utiliza algoritmos eficientes como:

- **Quicksort (por defecto)**
- **Mergesort** (estable, conserva el orden de elementos iguales)
- **Timsort** (en versiones modernas de Python, optimizado para datos parcialmente ordenados)

Estos algoritmos tienen una **complejidad promedio de $O(n \log n)$** , lo que los hace mucho más eficientes que los métodos cuadráticos ($O(n^2)$) como la ordenación por burbuja.

Ejemplo práctico:

```
df.sort_values('puntaje', ascending=False).head(5) # Mejores 5 puntajes
```

 En resumen:

Concepto	Beneficio en la práctica
DataFrame y Series	Permiten manejar datos con etiquetas y distintos tipos de variables de forma eficiente.
Vectorización	Reduce tiempos de ejecución al evitar bucles en Python.
Búsqueda y ordenamiento	Se realizan con algoritmos optimizados (en C) de complejidad $O(n)$ u $O(n \log n)$.